

Assignment 2 — Six-Hour Gate core

Build `six-hour-gate/alarm_triage.py` — three pure functions that decide whether a device alarm is worth waking someone for — and the tests that prove each rule. Nothing calls it yet, so you cannot break the live system.

Ishira Perera · Issued 28 Sep 2026 · Due 9 Oct 2026 · Supervisor: Gayan Priyanatha

The problem

Read `state/device_alarms_state.json` and `state/dessmonitor_device_alarms_state.json` before you write any code. Here is what they say today:

142	alarms tracked across 25 accounts and 21 plants
105	of them battery-voltage — 57 "too low", 26 "Low battery", 18 "too high", 4 charger-stops
121	already mailed 3× and permanently silenced, fixed or not

A battery crossing a voltage threshold is what a battery on an off-grid system does **every night**. It recharges the next morning. Meanwhile five records saying *"Battery connection is open"* — a genuinely dead pack — arrived in the same inbox, in the same format, at the same priority as the 57 that just got dark.

So the admin inbox is full and nobody acts on it. That is the failure. Not too few emails, too many worthless ones.

The gate applies one rule: **an alarm has to still be wrong six hours later**. Everything the fleet shouts about in between is recorded in state and never mailed. The number that matters is not how many emails we send — it's that when one arrives, someone gets in the van.

What you are building

One new file: `six-hour-gate/alarm_triage.py`. Three functions, all **pure** — they take data in and return data out. No file reads, no network, no email, no clock lookups buried inside them. Pass `now` in as an argument. That is what makes them testable, and it is why you cannot break anything.

FUNCTION	TAKES	RETURNS
<code>normalize(raw, vendor)</code>	one raw alarm dict from a vendor feed	the common record below
<code>classify(message)</code>	the vendor's message string	one of <code>COMMS</code> , <code>SOC</code> , <code>HARDWARE</code> , <code>LOAD</code> , <code>THERMAL</code> , <code>GRID</code> , <code>PV</code> , <code>UNKNOWN</code>
<code>is_actionable(record, now, config)</code>	a normalized record, the current UTC time, the loaded policy	<code>(True, None)</code> or <code>(False, "<reason>")</code>

The common record every vendor collapses to:

```
{
  "plant":      "Imbulgoda 3kW",
  "device":     "PN12345",
  "vendor_code": "1015",
  "message":    "Battery voltage is too low",
  "class":      "SOC",
  "started_at_utc": "2026-09-27T16:30:00+00:00",
  "open":       true,
  "customer":   "Gayan-IMH"
}
```

The gate never learns a vendor's name. ShineMonitor and DessMonitor both give you `gts` and `status`; Solis gives `alarmBeginTime` as epoch milliseconds and `state.normalize()` is the only place that difference exists.

All policy — dwell hours, the daylight window, which messages map to which class — comes from `six-hour-gate/config/alarm_triage.json`, which already exists. **Read it before you start.** Do not hard-code a 6, a 9, a 15 or a message pattern in your Python. If you find yourself typing a number that's in that file, you're writing the wrong line.

The rules

Full reasoning is in `six-hour-gate/README.md`. Read it. These are the five your code must satisfy.

R1 — age comes from the vendor's clock, not ours. Dwell is `now - alarm_start`, where `alarm_start` is the vendor's own timestamp. An alarm that was already 12 hours old when we first polled it counts as 12 hours old, not zero. Our `first_seen` is a fallback only, for vendors that give us nothing better.

R2 — convert the timestamp before you compare it. **WRITE THIS TEST FIRST** `gts` arrives in plant-local time, UTC+5:30. The GitHub runner is UTC. Subtract one from the other naively and every alarm looks **5.5 hours younger than it is**, so your six-hour gate quietly behaves as an eleven-and-a-half-hour gate. Nothing crashes. No test fails unless you write one. Alarms just silently stop escalating and everyone assumes the fleet got healthier.

This is the most important line in the assignment. Write the test for R2 first, before the implementation, and watch it fail against a naive version.

R3 — only count alarms that are actually open. `status` false or 0; for Solis, `state == 1`. A resolved alarm never escalates however old it is. The fetch scripts still run in TESTING MODE and return resolved alarms, so your function must not assume the feed is clean.

R5 — SOC alarms are judged on daylight, not a clock. 74% of our volume is battery voltage. "Too low" at 22:00 clears at 07:00 — nine hours, which a plain six-hour clock would escalate, on every off-grid plant, every night, forever. So an `SOC` alarm escalates only if it is still open through the local 09:00–15:00 window. A healthy system recharges in that window; one that doesn't has a real fault.

R6 — grid loss is on the clock like everything else. A 2–4 hour CEB cut never reaches an inbox. What outlives six hours is usually a tripped AC input breaker or a blown input fuse — our call-out, not the utility's.

COMMS — never mailed, at any age. A dark datalogger is the customer's network, not our equipment. It surfaces elsewhere: an offline logger writes `0.0000` rows, which trips the 3-day production-loss RED alert on the CSV side. That is deliberate, not an oversight.

Anything your matcher doesn't recognise becomes `UNKNOWN` and takes the plain six-hour clock. Letting a novel fault through is the safe failure; swallowing one is not. Log every `UNKNOWN` so the map can grow from real traffic.

The tests

New file: `src/test/java/org/ktronics/scripts/integration/test_alarm_triage.py`, following the pattern of the suites already there. Every test is named for the rule it defends, so a failure tells you which promise broke.

TEST	PASSES WHEN
<code>test_r1_escalates_just_past_six_hours</code>	open at 6 h 01 m escalates; at 5 h 59 m it does not
<code>test_r1_absent_from_feed_means_cleared</code>	an alarm missing from this poll is treated as cleared, not aged
<code>test_r2_plant_local_gts_against_utc_now</code>	a <code>gts</code> in UTC+5:30 compared to a UTC <code>now</code> yields the true age
<code>test_r3_handled_alarm_never_escalates</code>	<code>status</code> true / <code>state != 1</code> never escalates, at any age
<code>test_comms_never_mailed</code>	a COMMS alarm is suppressed at 1 h, 10 h and 100 h
<code>test_r5_soc_overnight_does_not_escalate</code>	open 22:00→07:00 local stays silent
<code>test_r5_soc_through_midday_escalates</code>	open 09:00→15:00 local escalates
<code>test_r6_short_grid_cut_silent</code>	a 3-hour grid outage is silent
<code>test_r6_long_grid_cut_escalates</code>	a 7-hour one escalates
<code>test_unknown_message_falls_back_to_clock</code>	an unmatched message becomes <code>UNKNOWN</code> on the 6 h clock, and is logged
<code>test_longest_pattern_wins</code>	"overload time out" classifies as LOAD via the longer pattern, not the shorter "overload"

Two hard requirements:

1. **The existing 35 UC3 tests must still pass, untouched.** The gate is additive. If you changed one of those tests to make yours pass, you broke something and hid it. Run the full suite before you push.
2. **Write `test_r2...` first, and watch it fail** against a naive implementation before you fix it. If it passes against the naive version, the test is wrong — not the code.

For R5, build your test times as explicit local datetimes with a fixed offset. Do not use `datetime.now()` anywhere in a test: a test that behaves differently at 3 p.m. than at 3 a.m. is worse than no test.

Out of scope

The README lists seven steps. You are doing part of step 2, and only the pure part. **Do not touch any of these:**

- `generate_device_alarms.py` — wiring the gate in is a separate change, after your functions are reviewed
- `check_device_alarms.sh` / `check_dessmonitor_device_alarms.sh` — R3's status filter changes what the live 6x-daily job pulls, for every customer, the moment it merges
- any `.github/workflows/*.yaml` — including the hourly poll change
- `state/*.json` — the live state files. Read them all you like; write nothing
- the Solis adapter, and anything about SolaX

This is not busywork fencing. Nothing calls `alarm_triage.py` when you're done, which means your branch **cannot** silence a real alarm or mail a customer, however wrong the code is. That is the whole reason you get this task in week two rather than in month three. Wiring it in is the dangerous part, and we do that together.

If you think a rule in the README is wrong, say so in the PR. Do not quietly implement something different — and do not quietly implement it as written if you believe it's wrong. Both of those are worse than an argument.

How to submit, and one thing to fix first

Same flow as assignment 1: branch, push, pull request against `main`, assign it to me, don't merge it yourself.

```
git checkout main && git pull
git checkout -b six-hour-gate-core
# ... work ...
git push -u origin six-hour-gate-core
```

PR title: `Six-Hour Gate: alarm_triage core + rule tests`. In the description, paste the output of the full test run — both your new tests and the 35 UC3 ones.

Close assignment 1 before you start this. Four answers in that PR are still wrong: the `config.py` threshold claim, the Solis test path, `baseline_kwh`, and the `UC3-UC8` tag. Pointing at a line number is not answering a question. A thread closes when I close it.

And the habit behind those four matters more here than it did there. All of them were provable in ten seconds by opening the file; you wrote what the structure implied instead. In assignment 1 that produced a wrong sentence in a document. In this module it produces a gate that looks like it works, escalates nothing for five and a half hours longer than anyone believes, and tells no one. **Open the file. Run the code. Print the value.** If you can't check something, say so in the PR and ask — that is a good answer, and I will never mind it.

One more thing, and I mean this: your section 9 in assignment 1 was the best part of it. Asking why the folder structure says `java` when it holds Python, and asking what Azure does here when nothing does — that is

exactly the instinct this job needs. Keep it. Bring me a section 9 with this PR too.

Ktronics (Private) Limited · IoT platform · internship assignment 2 of the training programme under Schedule 1 of the Internship Training Agreement dated 21 September 2026. Design of record: six-hour-gate/README.md